



Name: Jaykishan Saharan

Course: BTech CSE-SY (Core)

Division: D

Roll No. :- 26

DSA PRACTICAL ASSIGNMENT 1

08/10
Title:

Write a program that uses non-recursive functions to perform the following searching operation for a key value in a given list of integers.

i. Linear Search

ii. Binary search

Objective:

- To understand the linear search and its implementation.
- To understand Binary Search and its implementation.

Outcomes:

- At the end of this lab, the students will be able to implement the above searching operations.
- Use of arrays to carry out linear and Binary search



Aim:

Write a C++ program to search an element in the array of N elements using linear and binary search.

Conclusion:

Thus, we have implemented in C++ linear and binary search using an array and tested it for the above set of elements for successful and ~~un~~ unsuccessful case.

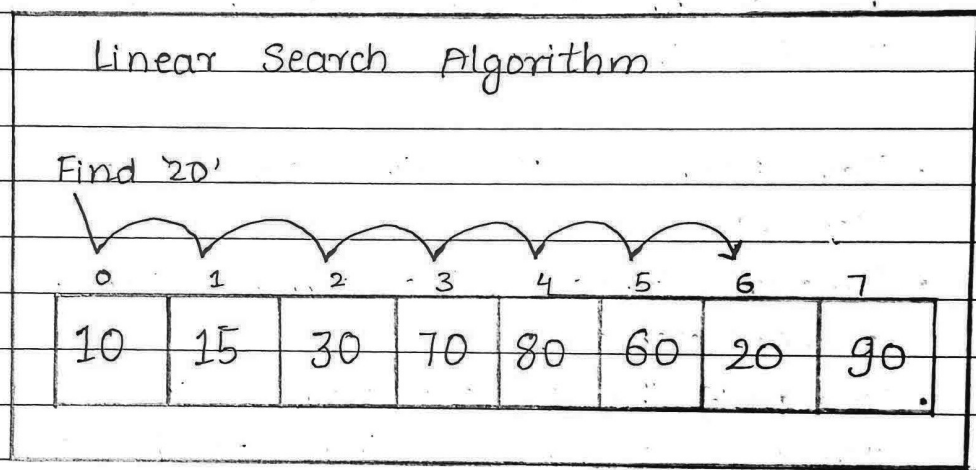
A
2/1/20

Questions

Q1 Explain Linear search with one example.

Sol Linear search is a method for searching for an element in a collection of elements. In linear search, each element of the collection is visited one by one in a sequential fashion to find the desired element. Linear search is known as sequential search.

Example:



Q2 Explain Binary search with one example.

Sol Binary search is a search algorithm used to find the position of a target value within a sorted array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty. The search interval is halved by comparing the target element with the middle value of the search space.



Conditions to apply Binary Search Algorithm in Data structure.

- The data structure must be sorted.
- Access to any external element of the data structure should take constant time.

Q3. Write an algorithm for linear search.

Sol. Algorithm for linear search:

The algorithm for linear search can be broken down into the following steps:

- Start: Begin at the first element of the collection of elements.
- Compare: Compare the current element with the desired element.
- Found: If the current element is equal to the desired element, return true or index to the current element.
- Move: Otherwise, move the next element in the collection.
- Repeat: Repeat steps 2-4 until we have reached the end of collection.
- Not found: If the end of the collection is reached without finding the desired element, it is not in the array.



Q4. Write an algorithm for binary search

Sol. Below is the step-by-step algorithm for Binary search.

- Divide the search space into two halves by finding the middle index "mid".
- Compare the middle element of the search space with the key.
- If the key is found at the middle element, the process is terminated.
- If the key is not found at the middle element, choose which half will be used as the next search space
 - If the key is smaller than the middle element, then the left side is used for the next search.
 - If the key is larger than the middle element, then the right side is used for the next search.
- This process is continued until the key is found or the total search space is exhausted.



Q5. Compare the time and space complexity of Linear and binary search.

Linear search

1. Works on unsorted or sorted data.
2. Best case : $O(1)$
3. Worst case : $O(n)$
4. Average case : $O(n)$
5. Space complexity : $O(1)$
6. Simple to implement

Binary search

1. Works only on sorted data.
2. Best case : $O(1)$
3. Worst case : $O(\log n)$
4. Average case : $O(\log n)$
5. Iterative : $O(1)$,
Recursive : $O(\log n)$
6. More efficient but require sorted input.